

Focus Monitor

Presentation



Index

- 01 주제 설명
- 02 시연
- 03 SW 설계
- 04 트러블 슈팅
- 05 팀원 소개
- 06 Q&A



주제 설명

최근 교육 시장이 전체 학습 과정을 관리하고 분석하는 방향으로 발전

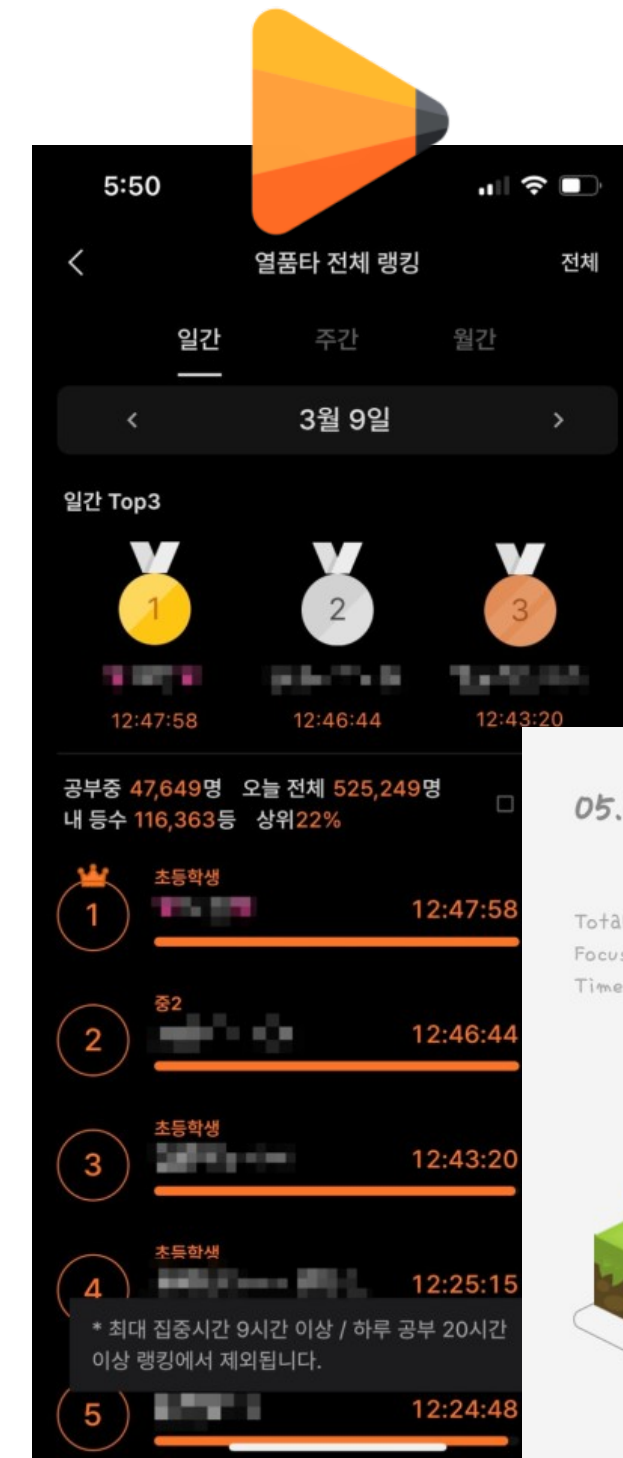
"집중"을 어떻게 정의?

(기존) 시간과 앱 사용 패턴 기반

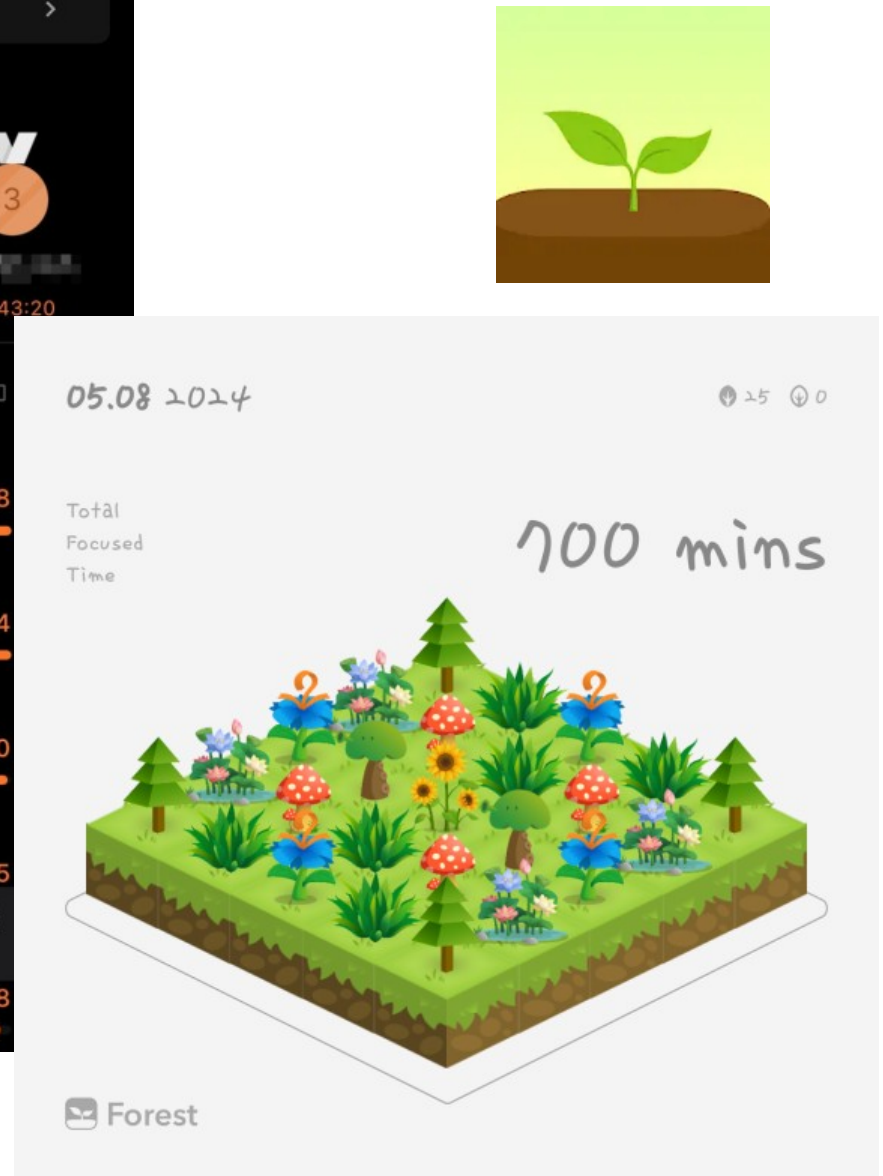


사용자의 실제 행동 기반

- 사용자가 자신의 집중도를 객관적으로 파악
- 졸음·휴대폰 사용·자리 비움이 감지될 때마다 즉각 피드백, 사용자의 행동 변화를 유도



열품타 : 열정 품은 타이머



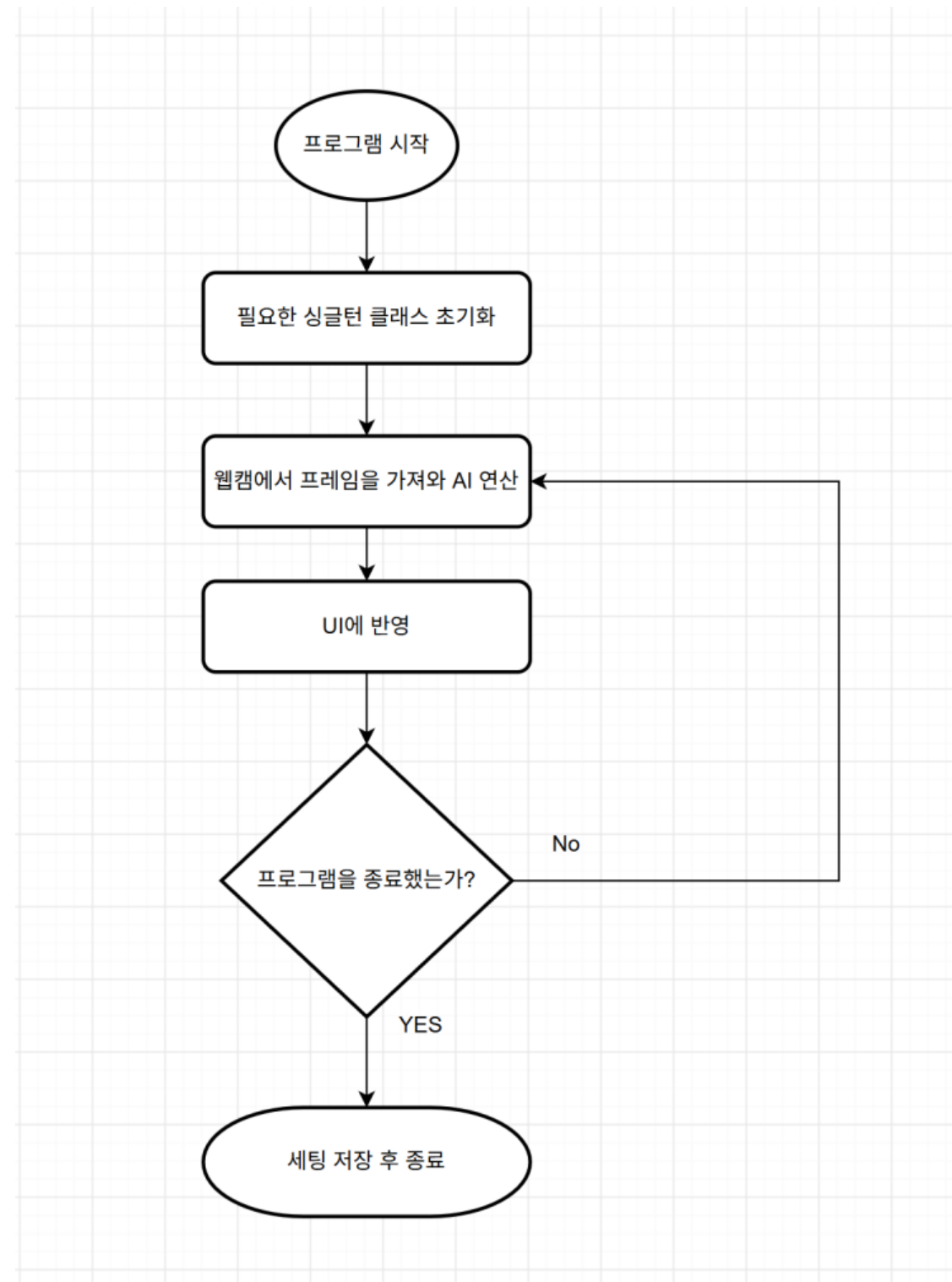
Forest

프로그램 시연



SW 설계

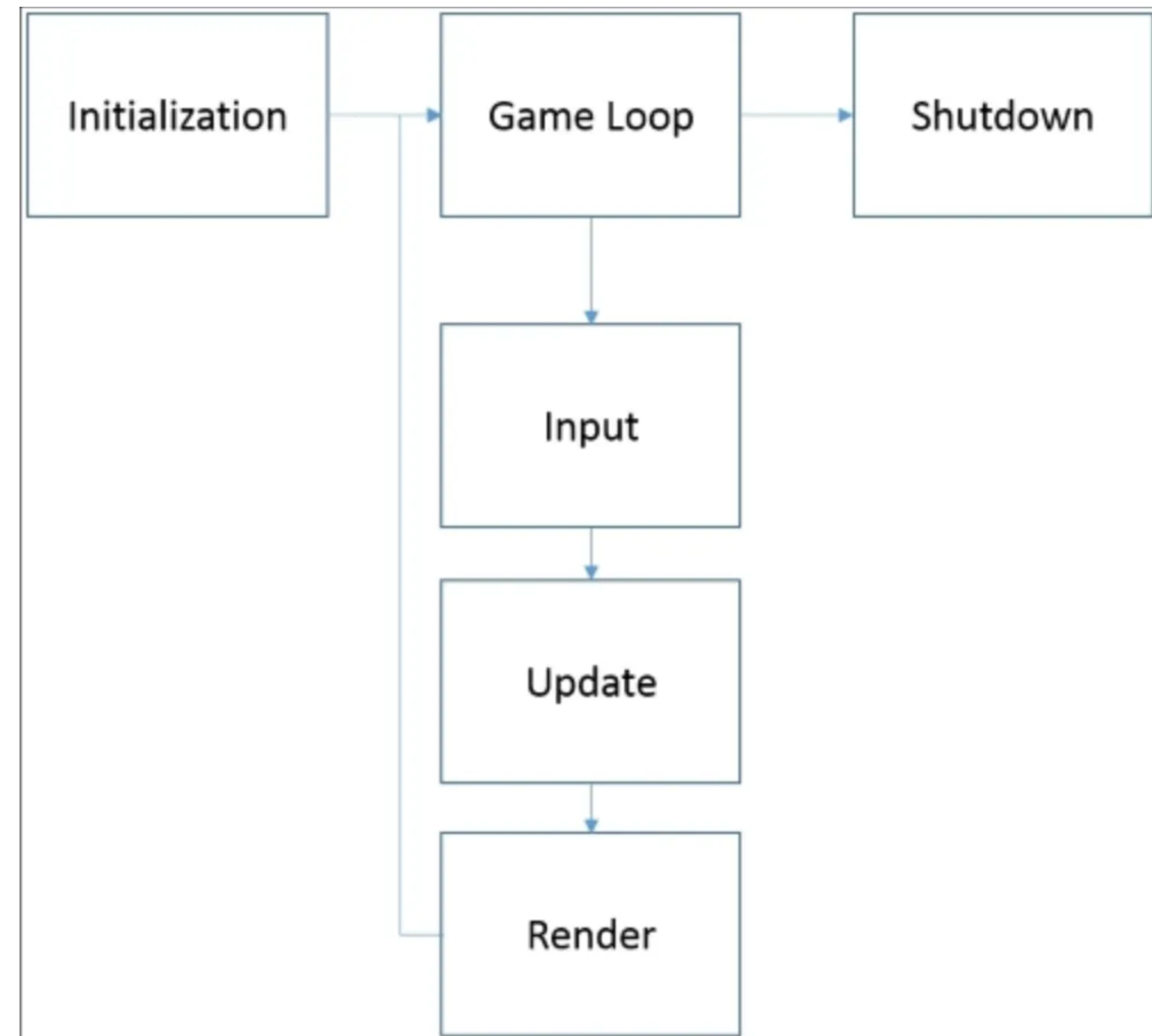
전체적인 실행 흐름도



실행 사이클로 게임루프 패턴 채용

내용은 간단합니다.

1. 초기 환경 설정
2. 루프를 탄다
3. 업데이트 (인풋, AI 등등 처리)
4. 렌더링 (UI, 카메라)



출처 : <https://www.meshy.ai/ko/blog/game-loop>

초기 환경 설정

```
def initialize(self) -> None:
    """Initialize and show application resources once."""

    System.FunctionLibrary.log("Application is starting...", System.LogLevel.NONE)
    settings_instance.load()
    report_manager.clear()
    report_manager.initialize()

    self._detection_pipeline = DetectionPipeline()
    input_manager.initialize(self._qt_app)
    bgm_manager.initialize(self._qt_app)
    effect_sound.initialize(self._qt_app)
    self._ui.initialize()
```

루프를 탄다

```
def __tick(self) -> None:
    if not self._running:
        return

    try:
        self.__update()
        self.__render()
    except Exception:
        self.stop()
        raise
    finally:
        input_manager.end_frame()
```

업데이트 (인풋 처리 및 AI 감지 영역 처리)

```
def __update(self) -> None:
    if not DEBUG:
        return

    boxes, texts = self.__build_debug_draw_data()
    camera_manager.draw_debug_frame(boxes, texts)
```

렌더링

```
def __render(self) -> None:  
    """Render the UI."""  
    self._ui.render()
```

공용으로 쓰는 클래스는 싱글턴 패턴

싱글턴 패턴은 프로그램에서
오직 하나의 객체만 생성하도록 보장하고,
어디서든 그 객체에 접근할 수 있게 해주는 디자인 패턴입니다.

Timer : 전역 타이머 콜백 및 틱 관리

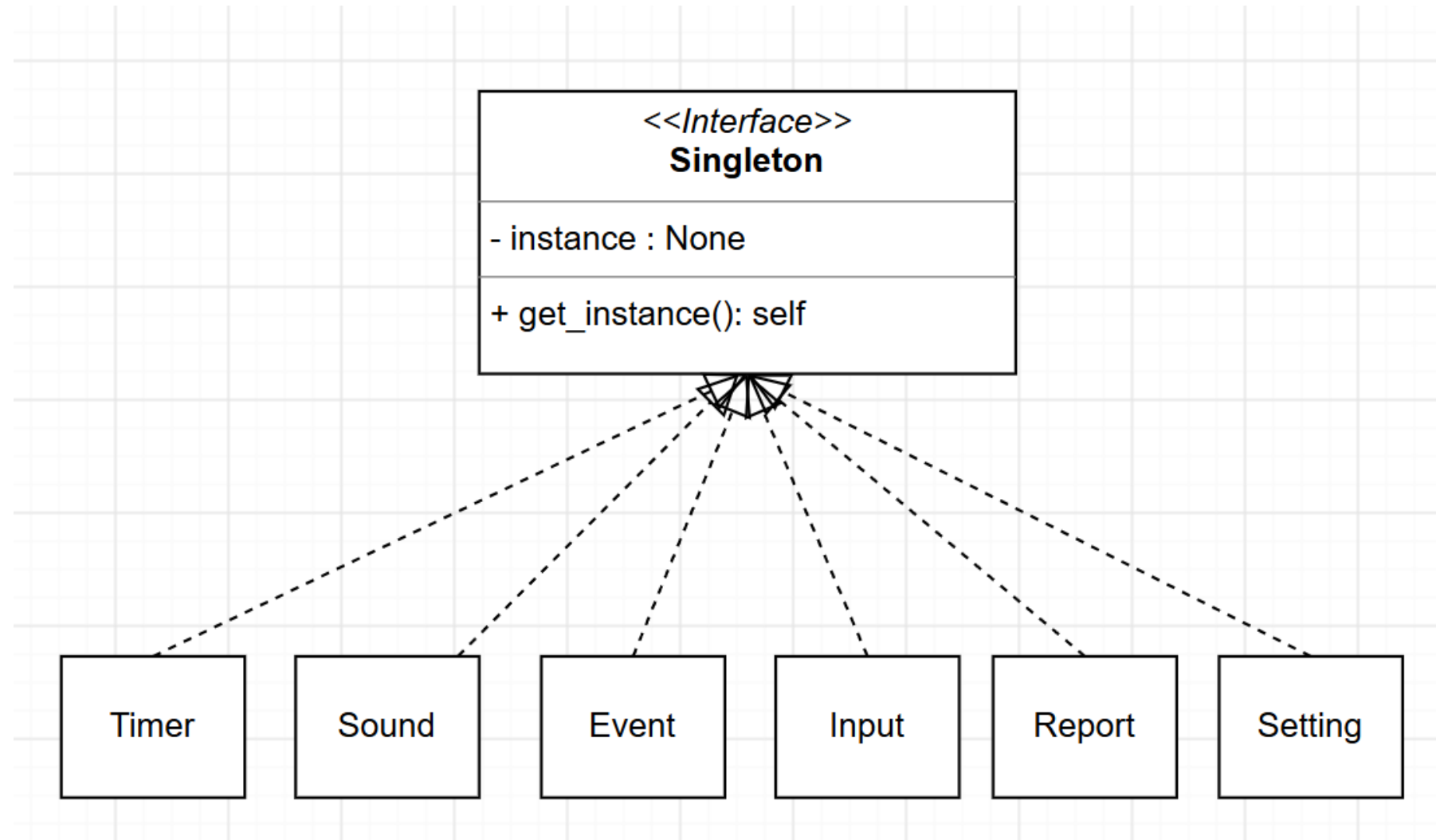
Sound : 사운드 실행 및 리소스 관리

Event : 이벤트 관련 콜백함수 관리

Input : 인풋처리 관련 관리

Report : 기록된 데이터 관리

Setting : 세팅 데이터 관리



싱글턴(이벤트 매니저)

싱글턴 클래스를 상속한다

```
Payload = dict[str, Any]
EventHandler = Callable[[Payload], None]
StoredEventHandler = EventHandler | WeakMethod
```

```
Samana, 4 days ago | 1 author (Samana)
class EventManager(Singleton):
    """Publish payloads to handlers registered under a string event key.
```

Usage:

```
def on_detected(payload: Payload) -> None:
    print(payload["confidence"])
```

```
event_manager.subscribe("drowsy_detected", on_detected)
event_manager.publish("drowsy_detected", confidence=0.95)
event_manager.unsubscribe("drowsy_detected", on_detected)
```

```
"""
```

```
def __init__(self) -> None:
    if getattr(self, "_initialized", False):
        return
```

```
self._initialized = True
self._subscribers: dict[str, list[StoredEventHandler]] = defaultdict(list)
```

새로운 클래스가 생성되지 않도록 막는다

```
event_manager: EventManager = EventManager.get_instance()
```

따로 간편하게 쓸 수 있도록 미리 변수를 생성

싱글턴(이벤트 매니저)

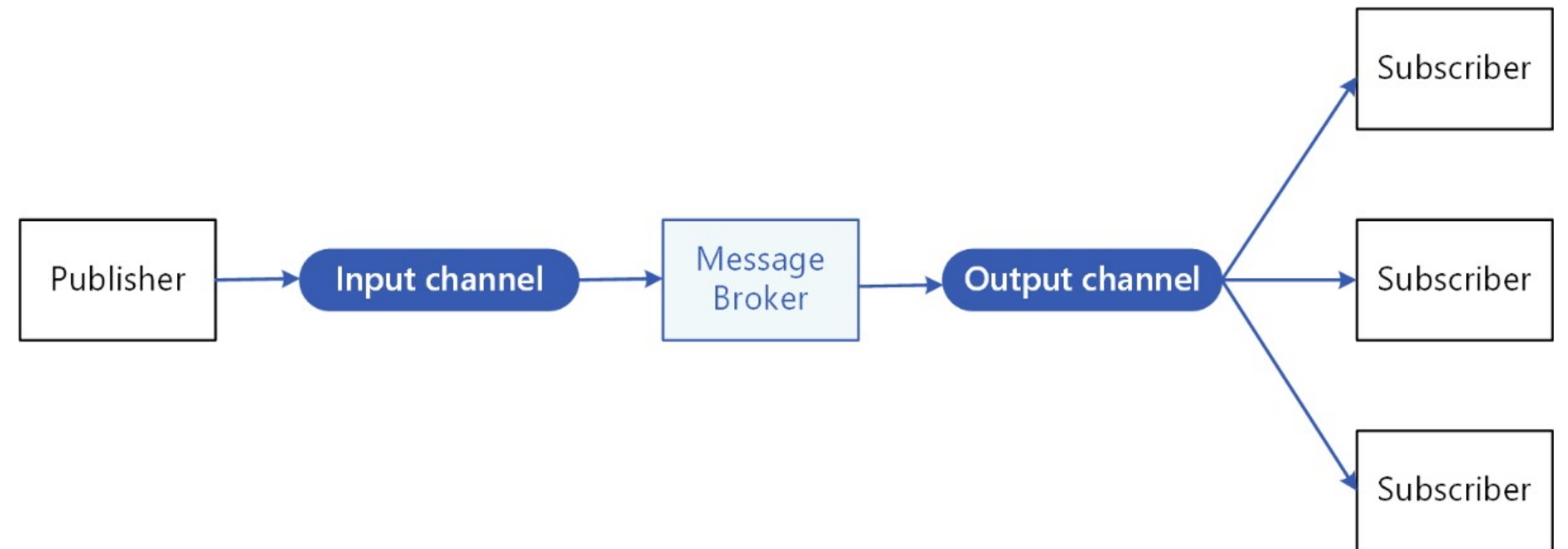
어느 클래스에서나 모듈만 불러오면
가져와서 쓸 수 있음

```
def start_requested(self, checked: bool = False) -> None:
    if self._is_started:
        return

    event_manager.publish(EventKey.START_REQUESTED.value)
    self.__show_camera_waiting()
    self.__set_started(self.__start_camera())
```

이벤트 관리는 구독패턴으로

구독 패턴은 데이터의 변경 사항을 지속적으로 감지하고, 상태가 바뀔 때마다 이를 등록된 구독자에게 자동으로 알림·전달하는 디자인/설계 패턴입니다.



출처 :<https://learn.microsoft.com/ko-kr/azure/architecture/patterns/publisher-subscriber>

구독패턴 (이벤트 매니저)

구독을 요청하는 함수
패러미터로 키 값과 콜백 함수를 받음

```
def subscribe(self, key: str, handler: EventHandler) -> None:
    """Register a handler for an event key."""
    stored_handler = self.__store_handler(handler)

    if stored_handler not in self._subscribers[key]:
        self._subscribers[key].append(stored_handler)
```

호출자 쪽에서는 이벤트 키 값과
이벤트 발생 시 호출되어야 하는 함수를 넘김

```
def __subscribe_events(self) -> None:
    event_manager.subscribe(EventKey.ABSENCE_DETECTED.value, self.__on_absence_detected)
    event_manager.subscribe(EventKey.DROWSY_DETECTED.value, self.__on_drowsy_detected)
    event_manager.subscribe(EventKey.PHONE_DETECTED.value, self.__on_phone_detected)
    event_manager.subscribe(EventKey.ALERT_CLEARED.value, self.__on_alert_cleared)
    event_manager.subscribe(EventKey.START_REQUESTED.value, self.__on_start_requested)
    event_manager.subscribe(EventKey.BREAK_REQUESTED.value, self.__on_break_requested)
```

구독패턴 (이벤트 매니저)

구독자에게 이벤트를 알리는 함수입니다.

해당 함수 호출 시

특정 키에 구독된 함수들에게 페이로드를 넘깁니다.

특정 키값으로 구독된 대상리스트를 가져옴

혹시라도 메모리에서

소멸된 객체라면 호출대상에서 제외

구독된 대상에게 데이터를 넘김

```
def publish(self, key: str, **payload: Any) -> None:
    """
        사용자가 지정한 이벤트 키에 대해 등록된 모든 핸들러를 호출하고, 페이로드를 전달합니다.

        Args:
            key (str): 이벤트 키
            payload (dict): 이벤트 핸들러에 전달할 페이로드
    """
    handlers: list[EventHandler] = []

    stored_handlers: list[StoredEventHandler] | None = self._subscribers.get(key)
    if not stored_handlers:
        return

    live_handlers: list[StoredEventHandler] = []
    for stored_handler in stored_handlers:
        handler = self.__resolve_handler(stored_handler)

        if handler is not None:
            # WeakMethod가 유효한 경우에만 핸들러를 호출하도록 합니다.
            handlers.append(handler)
            # WeakMethod를 보관하기 위해 live_handlers에 추가합니다.
            live_handlers.append(stored_handler)

    if live_handlers:
        self._subscribers[key] = live_handlers
    else:
        del self._subscribers[key]

    for handler in handlers:
        handler(payload)
```

실시간 집중도 감지

한 프레임에 대해서 다음 순서대로 감지

자리 이탈(AbsenceDetector) → 눈 감김 (EyeDetector) → 휴대폰 사용 (PhoneDetector)

①

②

③

```
# Create FrameContext
```

```
context = FrameContext(  
    frame=frame,  
    yolo_results=shared_yolo_results  
)
```

①

```
# Detect absence and save detected face bbox  
absence_result = self._absence_detector.detect(context)  
context.face_bbox = self._absence_detector.face_bbox
```

②

```
# Detect eye closedness and phone use
```

```
eye_result = self._eye_detector.detect(context)
```

③

```
phone_result = self._phone_detector.detect(context)
```

```
return [absence_result, eye_result, phone_result]
```

AbsenceDetector : InsightFace (buffalo_L), YOLO11n

: 사용자가 자리를 이탈했는지 검사

EyeDetector : MediaPipe (face landmarker)

: 사용자의 눈이 감겨있는지 검사

: face landmarker에서 눈 랜드마크를 찾아 EAR 계산

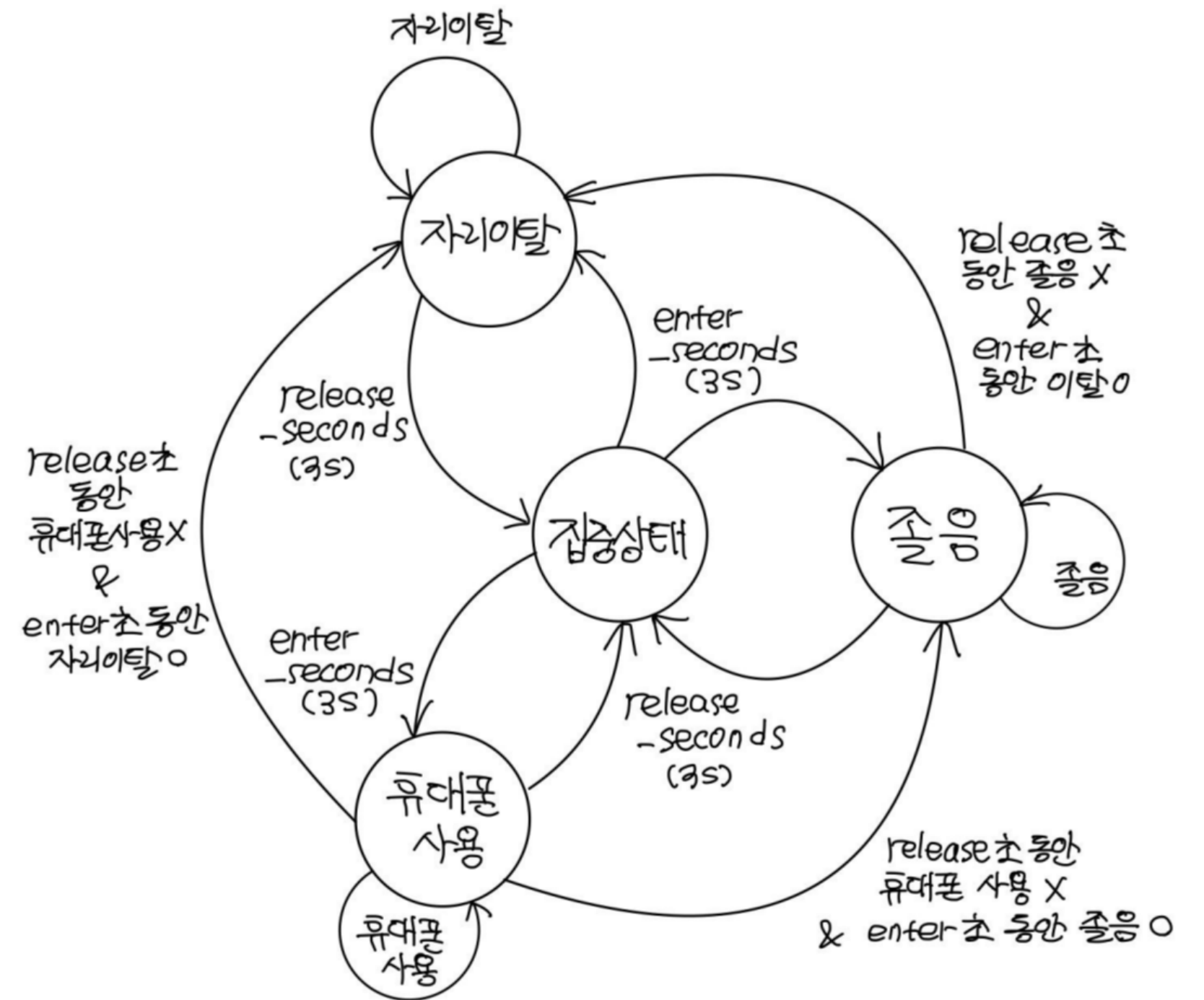
PhoneDetector : YOLO11n

: 사용자가 핸드폰을 사용하는지 검사

: 얼굴과 거리가 충분히 가까운 핸드폰이 존재하면 핸드폰을 사용하는 것

각 Detector의 결과를 받아 상태를 업데이트

- 자리 이탈 > 졸음 > 휴대폰 사용 > 집중 우선순위로 상태 감지
- frame 기반 (x) 초 기반 (O)
- 일정 시간 동안 감지 결과가 유지되어야 상태가 변함
- 상태가 변함을 이벤트 핸들러에 publish





트러블 슈팅

자리 이탈 인식 개선

자리 이탈 (Absence)

- 사용자가 아닌 얼굴을 찾았다고 해서 "자리에 있음"으로 처리하면 안됨
최초 1회 사용자의 얼굴을 등록하고 유사도가 높은 얼굴만 사용자로 인식
- 필기 등으로 사용자의 얼굴을 인식할 수 없다고 해서 "자리에 없음"으로 처리하면 안됨
얼굴을 인식할 수 없다면 이제껏 인식한 얼굴과 가깝고 충분히 큰 "사람"이 있는지 검사

프레임 개선을 위한 노력

- **PySide6 웹캠 (Left)**

PySide6 QVideoWidget으로
바로 출력

- **OpenCV 변환 (Right)**

OpenCv에서 받아온 웹캠 프레임을
PySide6 QImage로 변환하여 출력함

CPU 기반 AI VS GPU 기반 AI

Problems

Output

Debug Console

Terminal

Ports

GitLens 3

0: 480x640 1 person, 27.4ms
Speed: 0.9ms preprocess, 27.4ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 27.6ms
Speed: 1.0ms preprocess, 27.6ms inference, 0.4ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 23.3ms
Speed: 0.9ms preprocess, 23.3ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 22.5ms
Speed: 1.1ms preprocess, 22.5ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 23.3ms
Speed: 0.8ms preprocess, 23.3ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 27.6ms
Speed: 9.1ms preprocess, 27.6ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 26.5ms
Speed: 0.9ms preprocess, 26.5ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 32.7ms
Speed: 1.1ms preprocess, 32.7ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 33.8ms
Speed: 1.0ms preprocess, 33.8ms inference, 0.6ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 1 person, 30.5ms
Speed: 3.6ms preprocess, 30.5ms inference, 0.5ms postprocess per image at shape (1, 3, 480, 640)

パフォーマンス

新しいタスクを実行する

...

CPU

98% 4.92 GHz

メモリ

20.2/61.7 GB (33%)

ディスク 0 (C:)

SSD (NVMe)

2%

ディスク 1 (D:)

SSD (NVMe)

0%

Wi-Fi

Wi-Fi

送信: 64.0 受信: 88.0 KiB

GPU 0

AMD Radeon(TM) Gra...

0% (41 °C)

GPU 1

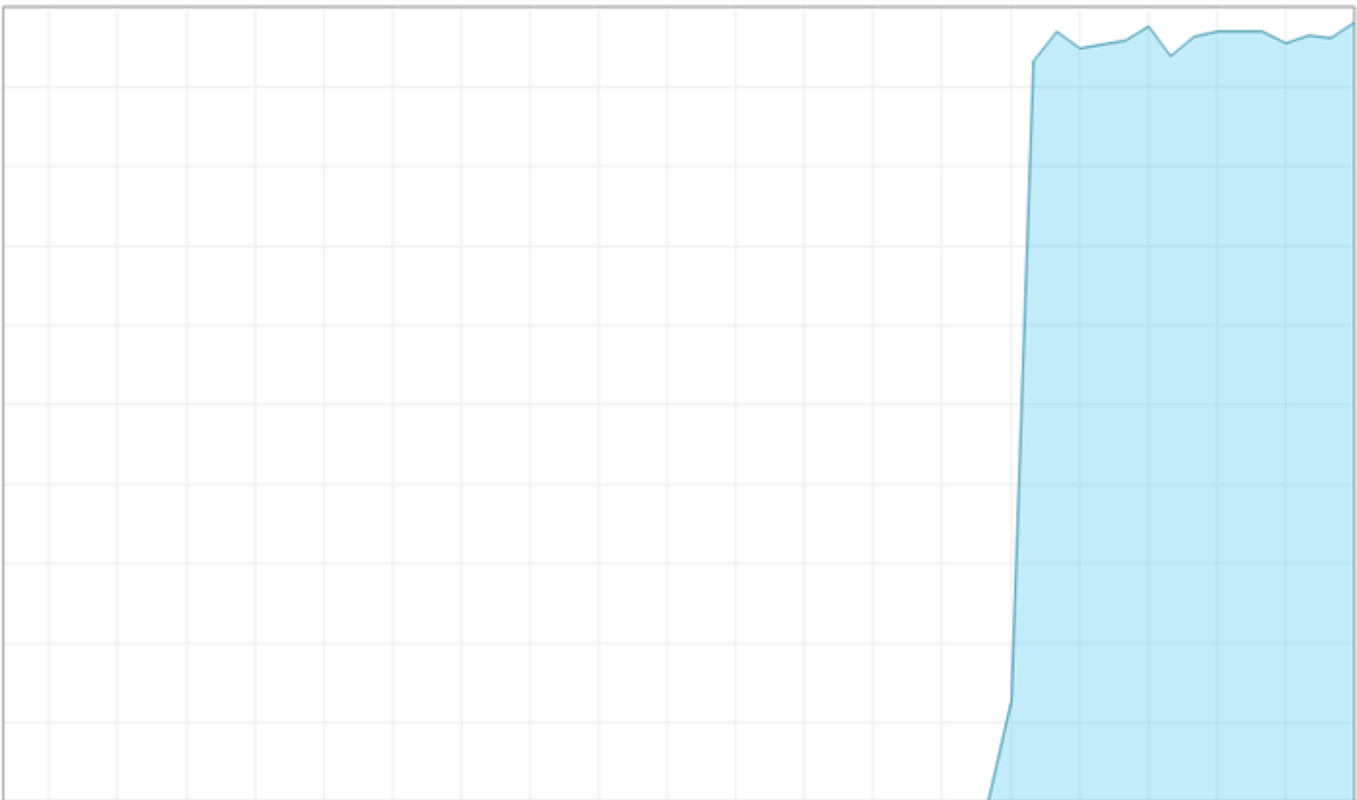
NVIDIA GeForce RTX ...

1% (39 °C)

CPU

AMD Ryzen 7 9800X3D 8-Core Processor

使用率



60 秒

使用率

98%

速度

4.92 GHz

基本速度:

4.70 GHz

ソケット:

1

コア:

8

論理プロセッサ数:

16

仮想化:

有効

L1 キャッシュ:

640 KB

L2 キャッシュ:

8.0 MB

L3 キャッシュ:

96.0 MB

プロセス数

281

スレッド数

6125

ハンドル数

141853

稼働時間

0:10:10:00

CPU 기반 AI VS GPU 기반 AI

Problems

Output

Debug Console

Terminal

Ports

GitLens 5

0: 480x640 (no detections), 4.4ms
Speed: 0.8ms preprocess, 4.4ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 5.0ms
Speed: 0.5ms preprocess, 5.0ms inference, 0.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 6.7ms
Speed: 0.8ms preprocess, 6.7ms inference, 1.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 5.7ms
Speed: 0.7ms preprocess, 5.7ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 5.6ms
Speed: 0.7ms preprocess, 5.6ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.8ms
Speed: 0.6ms preprocess, 4.8ms inference, 1.0ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.8ms
Speed: 0.7ms preprocess, 4.8ms inference, 0.7ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.2ms
Speed: 0.8ms preprocess, 4.2ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.3ms
Speed: 0.6ms preprocess, 4.3ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.7ms
Speed: 0.7ms preprocess, 4.7ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 4.7ms
Speed: 0.8ms preprocess, 4.7ms inference, 0.8ms postprocess per image at shape (1, 3, 480, 640)

0: 480x640 (no detections), 5.2ms
Speed: 0.6ms preprocess, 5.2ms inference, 1.1ms postprocess per image at shape (1, 3, 480, 640)

パフォーマンス

新しいタスクを実行する

...

CPU

12% 4.91 GHz

メモリ

22.9/61.7 GB (37%)

ディスク 0 (C:)

SSD (NVMe)

2%

ディスク 1 (D:)

SSD (NVMe)

0%

Wi-Fi

Wi-Fi

送信: 0.2 受信: 20.0 Mt

GPU 0

AMD Radeon(TM) Gra...

0% (37 °C)

GPU 1

NVIDIA GeForce RTX ...

15% (39 °C)

GPU

NVIDIA GeForce RTX 5090

15% 3D

15% Copy

0%

0% Video Encode

0% Video Decode

1%

専用 GPU メモリ

31.5 GB

共有 GPU メモリ

45.7 GB

使用率

15%

GPU メモリ

4.7/77.2 GB

専用 GPU メモリ

4.5/31.5 GB

共有 GPU メモリ

0.3/45.7 GB

温度

39 °C

ドライバーのバージョン:

32.0.15.9636

ドライバーの日付:

2026-04-23

DirectX バージョン:

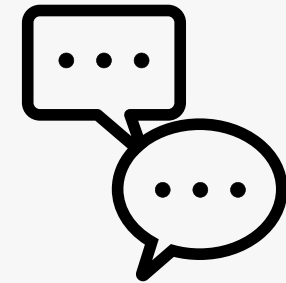
12 (FL 12.2)

物理的な場所:

PCI バス 1, デバイス 0, 機能 0

ハードウェア予約済みメモリ:

69.0 MB



질의응답 시간입니다.



Thank you